Using Timeout to Handle Failures in Microservices

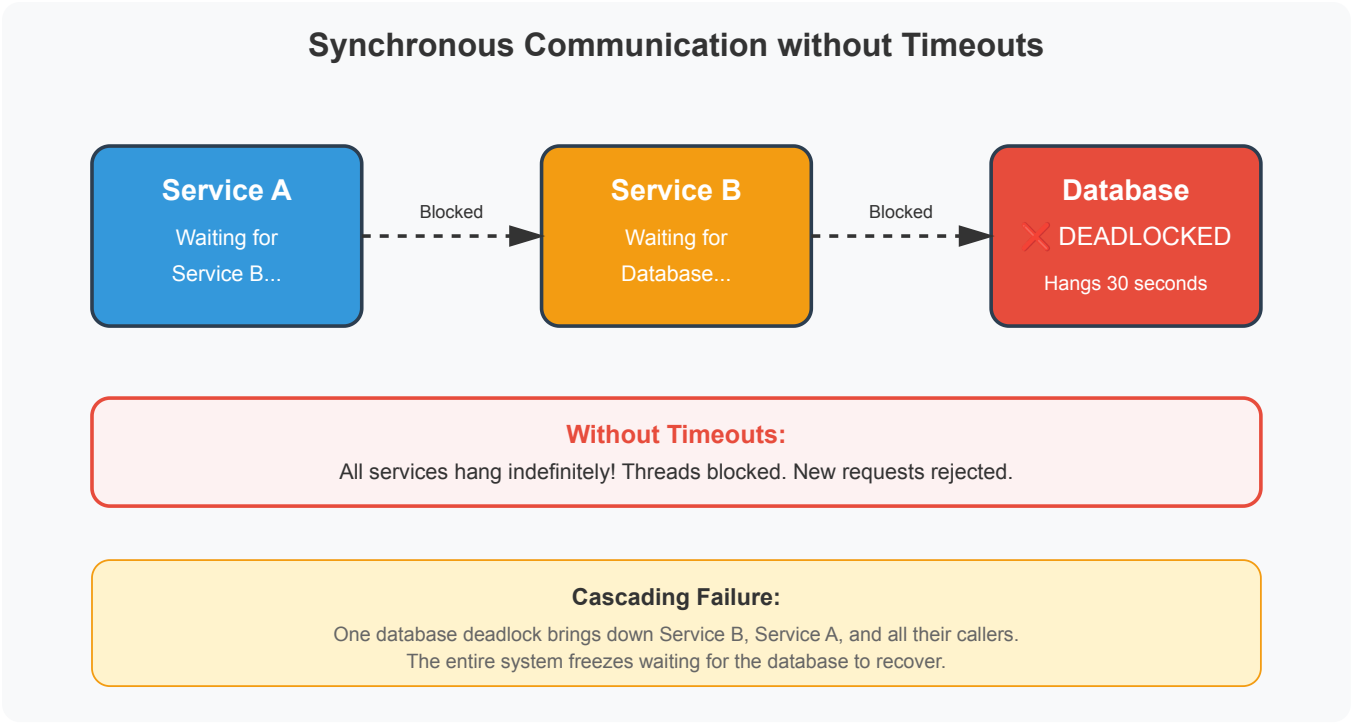
systemdesignschool.io/fundamentals/timeout

Timeout

We explored four patterns for handling synchronous communication failures. Timeouts are the simplest and most fundamental. Every synchronous call should have a timeout. Without one, a thread can block forever waiting for a response that never arrives.

What Timeouts Prevent

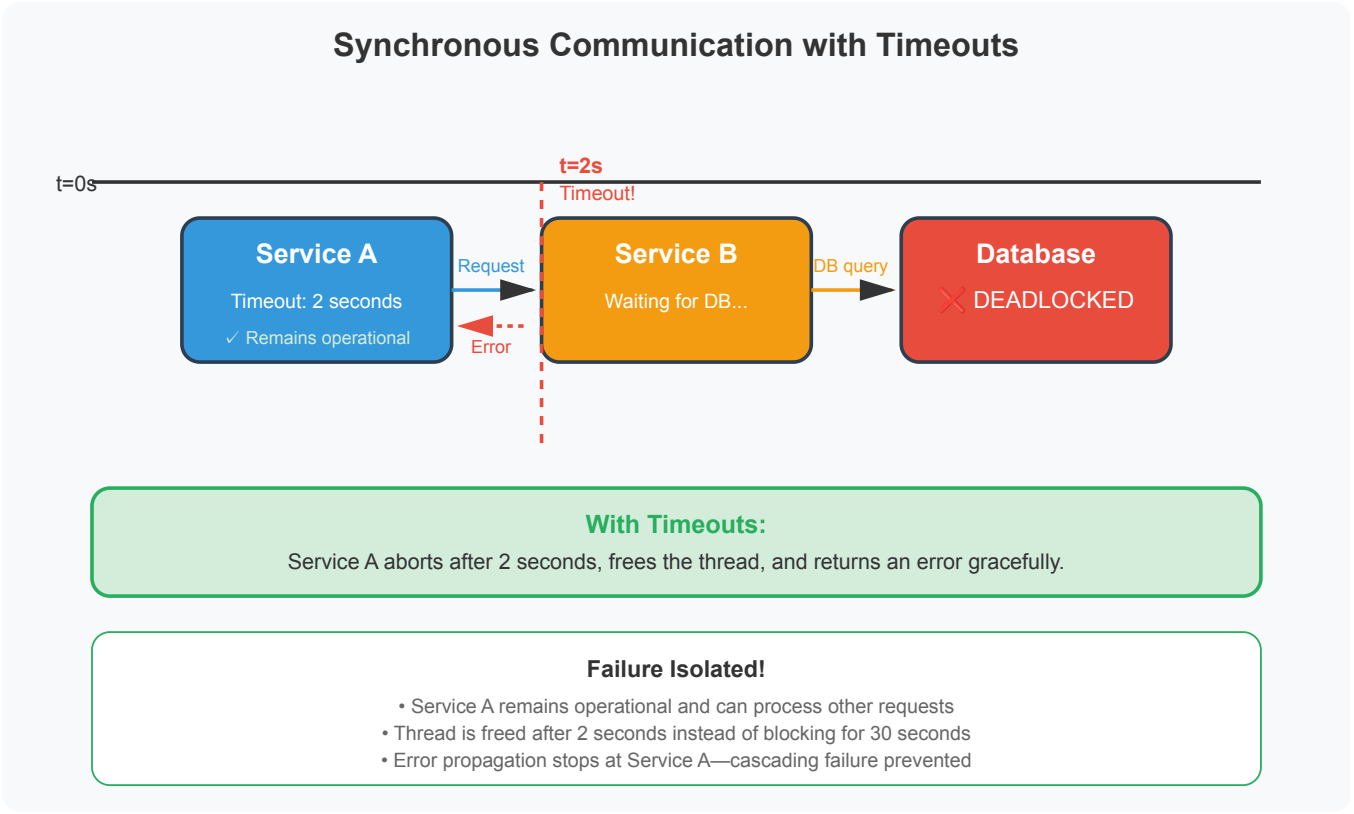
Service A calls Service B. Service B queries a database. The database deadlocks on a transaction and hangs for 30 seconds. Service B's thread blocks waiting for the query. Service A's thread blocks waiting for Service B. If 100 requests arrive at Service A, all 100 threads block. New requests cannot be processed. Service A appears to freeze entirely.



This cascading freeze spreads through the entire system. Service C calls Service A. Service C's threads block too. The failure of one database query brings down three services.

How Timeouts Work

A timeout sets a maximum wait duration. Service A sets a 2-second timeout when calling Service B. If Service B does not respond within 2 seconds, Service A aborts the request and returns an error. The thread becomes available for other work.



When the database deadlocks, Service B's threads block. But Service A only waits 2 seconds per request. After 2 seconds, Service A returns an error to its caller. Service A remains operational. It can process other requests that do not depend on Service B.

Setting Timeout Values

Choose timeouts based on measured latency under normal conditions. If Service B normally responds in 100ms, set a timeout around 500ms. This allows for some variance while preventing indefinite waits.

Measure the 95th or 99th percentile response time. If 99% of requests complete in 300ms, a 1-second timeout catches the slow 1% without being too aggressive. Too short a timeout causes false failures. Too long a timeout defeats the purpose.

Different operations need different timeouts. Reading from cache might timeout at 100ms. Complex database queries might need 5 seconds. External API calls might allow 10 seconds. Set timeouts per operation based on expected latency.

Implementing Timeouts

Most HTTP and RPC libraries support timeouts directly. In Python with requests:

```
import requests try: response =
requests.get("https://api.example.com/users/123", timeout=2) return
response.json() except requests.exceptions.Timeout: return {"error": "Service
unavailable"}
```

In Go with gRPC:

```
ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second) defer cancel() response, err := client.GetUser(ctx, &pb.UserRequest{Id: 123}) if err != nil { // Handle timeout or other error return nil, err }
```

Database connections also need timeouts. A PostgreSQL connection in Java:

```
Properties props = new Properties(); props.setProperty("connectTimeout", "2"); props.setProperty("socketTimeout", "5"); Connection conn = DriverManager.getConnection(url, props);
```

Timeouts and Retries

Timeouts work with retries. A request times out after 2 seconds. The caller retries with exponential backoff. Wait 1 second, retry. If that times out, wait 2 seconds, retry again. After 3 attempts, give up and return an error or fallback.

Without timeouts, retries make things worse. The first request blocks indefinitely. The retry also blocks. Now two threads are stuck instead of one. Timeouts ensure each retry attempt has a deadline.

The next section on retries covers retry strategies in detail. Circuit breakers build on timeouts by tracking failure rates and stopping requests to consistently failing services.